Internet of Things and Wireless Networks

Lab 3 Overview

Lab 2 summary

You ...

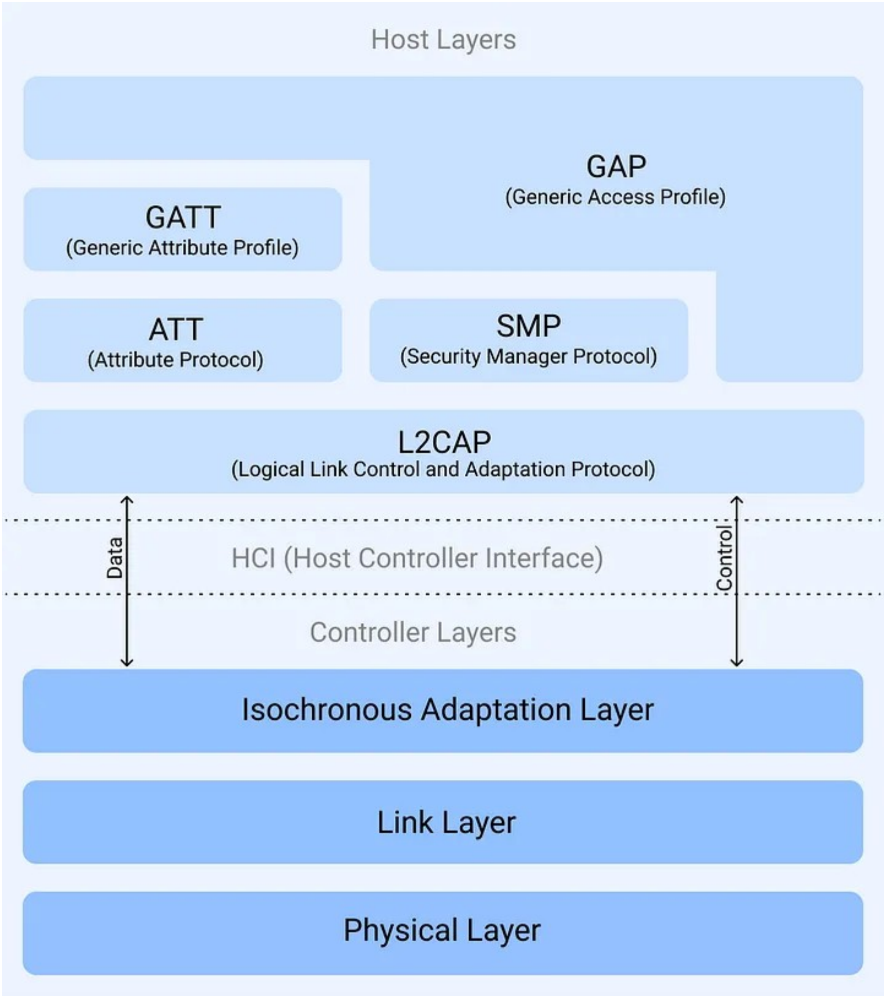
- implemented multi-hop communication,
- got experience with the challenges of data relaying.

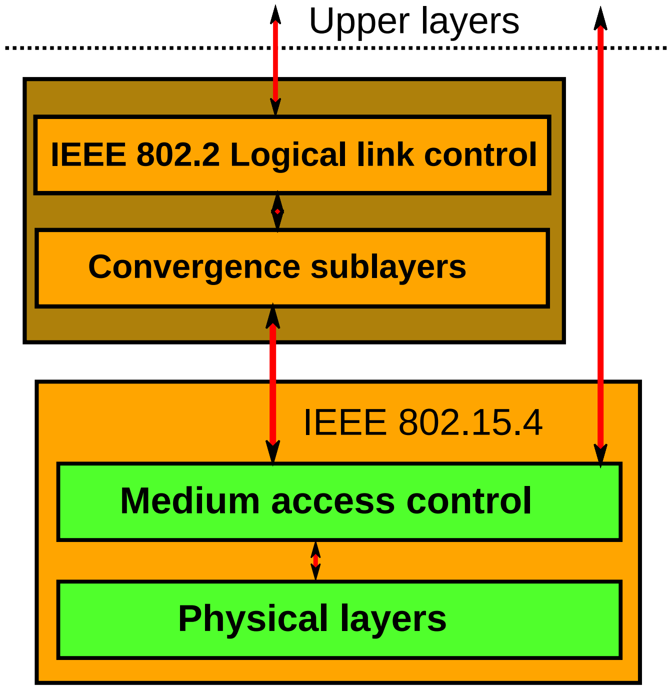
So far, you implemented applications using BLE (802.15.1).

Now, we want to look at 802.15.4 and IP based protocols.

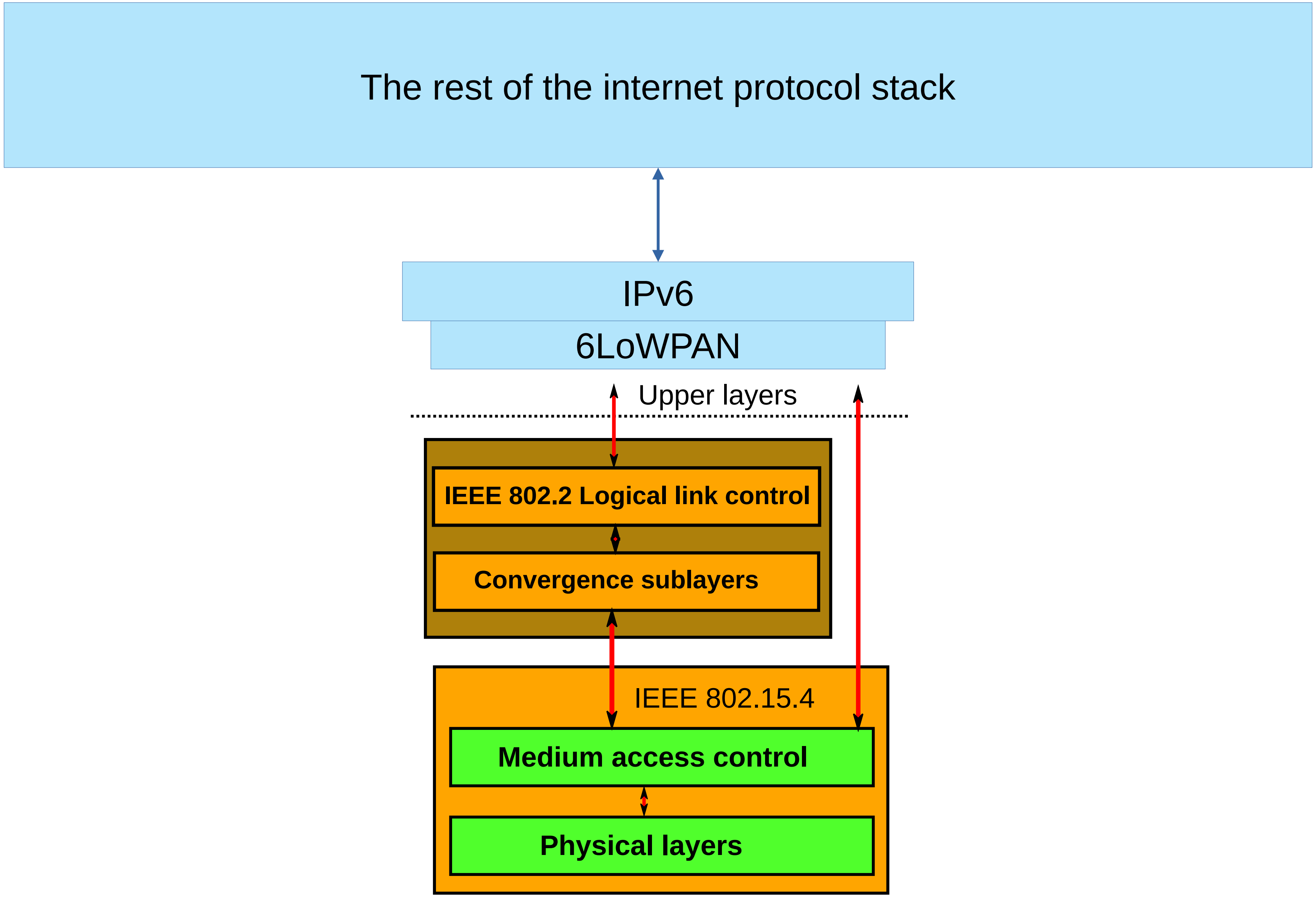
IEEE 802.15.4

BLE vs 802.15.4





Source: https://medium.com/@thomsmed/core-bluetooth-and-the-ble-stack-95ddc29c8671
Rob Blancoderivative Tsaitgaist - IEEE_802.15.4_protocol_stack.png, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=7385505


The rest of the internet protocol stack
IPv6
6LoWPAN
Upper layers
IEEE 802.2 Logical link control
Convergence sublayers
IEEE 802.15.4
Medium access control
Physical layers

IEEE 802.15.4 Network Topology

- Peer-to-Peer or Star Topology supported
  - With Peer-to-Peer, each node needs to know the PAN identifier and the frequency channel of the PAN
  - Peer-to-Peer is sufficient for the lab
  - Also, hardcoding PAN identifier and channel is fine.

IEEE 802.15.4 in Zephyr

Networking in Zephyr

- Similar to Linux
  - 802.15.4 is just a type of network interface
  - Configured via the Network Management API and the Network Configuration Library
  - Activate 802.15.4, IPv6, and 6Lo via Kconfig
- POSIX-Like sockets API

CoAP

Constrained Application Protocol

- Specialized web transfer protocol
- Based on UDP
- Similar to HTTP, but more lightweight
- Natively supported in Zephyr
  - You still need to do the socket management yourselfs

CoAP: Modes of Message Transmission

```
Client              Server
   |                   |
   |   CON [0x7d34]    |
   +------------------>|
   |                   |
   |   ACK [0x7d34]    |
   |<------------------+
   |                   |

 Figure 2: Reliable Message Transmission
```

```
Client              Server
   |                   |
   |   NON [0x01a0]    |
   +------------------>|
   |                   |

 Figure 3: Unreliable Message Transmission
```

Source: Z. Shelby, K. Hartke, and C. Bormann, „The Constrained Application Protocol“, RFC 7252

CoAP: Responses

```
Client              Server
  |                   |
  |   CON [0x7a10]    |
  | GET /temperature  |
  |   (Token 0x73)    |
  +----------------->|
  |                   |
  |   ACK [0x7a10]    |
  |<-----------------+
  |                   |
  ... Time Passes  ...
  |                   |
  |   CON [0x23bb]    |
  |   2.05 Content    |
  |   (Token 0x73)    |
  |     "22.5 C"      |
  |<-----------------+
  |                   |
  |   ACK [0x23bb]    |
  +----------------->|
  |                   |
```

Figure 5: A GET Request with a Separate Response

```
Client              Server     Client              Server
  |                   |          |                   |
  |   CON [0xbc90]    |          |   CON [0xbc91]    |
  | GET /temperature  |          | GET /temperature  |
  |   (Token 0x71)    |          |   (Token 0x72)    |
  +----------------->|          +----------------->|
  |                   |          |                   |
  |   ACK [0xbc90]    |          |   ACK [0xbc91]    |
  |   2.05 Content    |          |  4.04 Not Found   |
  |   (Token 0x71)    |          |   (Token 0x72)    |
  |     "22.5 C"      |          |    "Not found"    |
  |<-----------------+          |<-----------------+
  |                   |          |                   |
```

Figure 4: Two GET Requests with Piggybacked Responses

```
Client              Server
  |                   |
  |   NON [0x7a11]    |
  | GET /temperature  |
  |   (Token 0x74)    |
  +----------------->|
  |                   |
  |   NON [0x23bc]    |
  |   2.05 Content    |
  |   (Token 0x74)    |
  |     "22.5 C"      |
  |<-----------------+
  |                   |
```

Figure 6: A Request and a Response Carried in Non-confirmable Messages

Source: Z. Shelby, K. Hartke, and C. Bormann, „The Constrained Application Protocol“, RFC 7252

CoAP: Summary

- Lots of Design Choices
- Supports Reliable (ACKed) and Unreliable Messages
- Details: See RFC 7252

Deadline

Sunday, June 14th, 23:59

Some Hints

- You might not be able to set the extended 802.15.4 address if you use platformio. It will tell you it did, and in the process break everything.
- You may need to disable automatic interface configuration. And you may have to set static IPv6 addresses yourselfs
  - Alternatively: use link local adresses. But please don’t hard code them, maybe use link local multicast to discover them.
- Enabling Debug Logs for the networking subsystem and/or using the Zephyr Shell can help you with debugging.

Now: Lab 2 Presentations